

Proofs by Mathematical Induction on Lists

```
theory List_Induction
  imports Main
begin

thm append.simps

lemma “ $(xs @ ys) @ zs = (xs @ (ys @ zs))$ ”
proof(induction xs)
  case Nil
  have “ $([] @ ys) @ zs = ys @ zs$ ”
  apply (subst append.simps)
  ..
  also have “ $\dots = ([] @ (ys @ zs))$ ”
  apply (subst append.simps)
  ..
  finally show ?case
  .
next
  case (Cons x xs)
  have “ $((x \# xs) @ ys) @ zs = (x \# (xs @ ys)) @ zs$ ”
  apply (subst append.simps)
  ..

  also have “ $\dots = x \# ((xs @ ys) @ zs)$ ”
  apply (subst append.simps)
  ..
  also have “ $\dots = x \# (xs @ (ys @ zs))$ ”
  apply (subst Cons.IH)
  ..
  also have “ $\dots = (x \# xs) @ ys @ zs$ ”
  apply (subst append.simps)
  ..
  finally show ?case
  .
qed

end
```